Design and Implementation of a Line Following and Obstacle Avoidance Robot

Mohamed Amer*, Daria Shushkova*,Andrey Sysoev*, Sadim Rahman*
*Systems Engineering and Prototyping, BSc Electronic Engineering
Hochschule Hamm-Lippstadt, Lippstadt, Germany
{mohamed-ahmed-mohamed-ali.amer, daria.shushkova, andrey.sysoev, sadim-rahman.badhan}@stud.hshl.de

I. Introduction

This project focuses on the development of a simple mobile robot that can autonomously follow a black line on a white surface and avoid obstacles placed in its path. The robot uses infrared sensors to detect the line and an ultrasonic sensor to detect nearby objects. An Arduino microcontroller is used to process sensor inputs and control motors accordingly.

II. System Design and Engineering

The robot design was developed based on **tasks** and **milestones** defined at the beginning of the project. These included line following, obstacle detection and avoidance, and integration of all components into a working prototype. It uses two infrared (IR) sensors to detect the black line and an ultrasonic sensor to measure the distance from obstacles. These sensors send signals to an Arduino Uno, that acts as the main controller.

The Arduino processes data and decides how the system should respond: either adjusting its path to stay on the line or stopping and maneuvering around an obstacle. Movement is handled by two DC motors controlled through an L298N motor driver.

Power is provided by a 9V battery pack, with regulation to supply the required voltages for the microcontroller and sensors. A simplified block diagram of the system is shown in Fig. 1

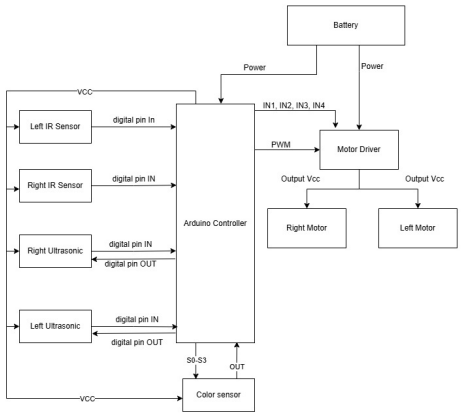

Fig. 1. Block diagram of the robot system architecture.

The vision of a system started with creating SysML diagrams to preserve and structure our project. It have the four Pillars of SysML:

- Requirements modeling (e.g., requirement diagram)
- Parametric modeling
- Structural modeling (e.g., Block Definition diagrams (BDD), Internal Block diagrams (IBD))
- Behavioral modeling (e.g., Use Case diagrams, Activity Diagrams, Sequence Diagrams, State Machine Diagrams)

1) **Requirement Diagram**
   It contains both **functional** and **nonfunctional** requirements of the autonomous vehicle system. These include navigation, obstacle handling, speed optimization, and route management, with their derived requirements and hardware components.
   See Fig 3 the Requirement diagram.

2) **Parametric Diagram**
   The **Parametric Diagram** define parametric relationships between properties of blocks. This section uses parametric diagrams to represent mathematical and physical constraints of system behavior. See Fig 4 the Parametric diagram.

3) **Block Definition Diagram**
   The **Block Definition Diagram (BDD)** provides a structural decomposition of the *Autonomous Vehicle System*. It represents the main logical blocks and how they are hierarchically related through composition relationships.

   The system is broken down into three primary subsystems:
   - **Chassis:** Includes physical mounting blocks such as the battery and breadboard holder, IR sensor holder, motor holder, and ultrasonic sensor holder.
   - **Controllers:** Main controller (*Arduino*) and the motor controller block responsible for driving the actuators.
   - **Components:** Wheels and motors, which are part of the physical actuation system.

   See Fig 5 the Block Definition diagram .

4) **Use Case Diagram**

Use case diagram models the interactions between **User** (the operator) and system functionalities. The diagram focuses on a *"happy path"* — **Drive Autonomously**, which includes supporting behaviors like:

- Adjust Speed
- Avoid Obstacle
- Follow Path

Extra behaviors such as **Drive in Oval** and **Drive in Figure Eight** are modeled as extensions of the routing logic.

See Fig 6 the Use Case diagram.

5) **State Machine Diagram**
The system begins in the "**stopped**" state. Upon initialization,it switches to the "**perform autonomous navigation**" state, where it drives left or right based on the input of the line-following sensor. If obstacle has been detected, system transitions to the "**obstacle_avoid**" state.

In "**obstacle_avoid**", the vehicle maneuvers around the obstacle:

- it goes "**backward**" until save distance to do a maneuver
- chooses path depending if where are walls on the "**left**" or "**right**"
- does "**line search**" to find the path after obstacle avoidance have been completed

See Fig 7 the State Machine diagram.

6) **Sequence Diagram**
The **Sequence Diagram** shows the interaction between the Operator, AVS Controller, sensors, and motor control during autonomous navigation. The controller continuously uses path and obstacle data to send commands to motors: `setForward()`, `setLeft()`, or `setRight()`.

If an obstacle is detected, the system enters **Obstacle Avoid Mode**, executing timed motor actions. An `alt` block handles conditional stopping or recovery based on search timeout and sensor feedback.

See Fig 8 the Sequence diagram.

III. Prototyping and Design

Our team was supplied with several Electronic components and a pre-designed chassis for development of the line follower vehicle. Our team was tasked to design a suitable mechanical design that would perform well in practice while utilizing all of the functionalities requested. A iterative design methodology was used to reach the final design made.

A. Electronic Components

As shown in Table IV a brief about the electronic components supplied and their purpose is mentioned. Key components that needed much attention were the infrared sensor, the battery and the motor driver. The Infrared sensor was a digital active high sensor.Thus, it was needed to be calibrated on the line we are following. Furthermore, dealing with the battery and the motor driver supplied, it was critical to ensure that voltages coming out of the motor driver voltage regulator are 5 volts. otherwise, this will expose the components to permanent damage. For safety purposes and avoiding any damaging to the components, we simulated the entire circuit in TinkerCad [2].

B. Virtual and Physical Prototypes

All of the prototyping design iterations were simulated in Solid works software. Our goal was to design as few parts as we can to avoid the given printing time limit. It was agreed on at the end to use the following parts followed by their main objective:

- **2x Motor mounts:** The motor mounts were used to mount the motors in a rigid way so it can support the entire weight of the vehicle.
- **2x IR sensor mounts:** The main objective is to keep the IR sensors too close to the ground to avoid lighting noise from the surrounding environment
- **2x Ultrasonic mounts:** Really lightweight components just to fix the ultrasonic without causing a high penality in printing time
- **battery and breadboard mount:** It function is to be multifunctional part holding the breadboard on its upper side and securing the battery underneath it

Our first print trial was not so successful at the beginning due to unaccounted error in our design. Then the parts were modified to have some fit tolerance to account for the error resulting from the 3D printer. Moreover on design iterations, we included slots instead of holes for the IR sensors so their position can be easily modified if needed. A visualization of the final prototype reached is shown in Figure 2.

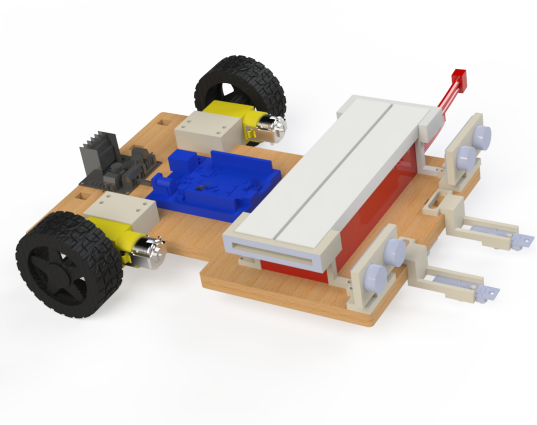

Fig. 2. Block diagram of the line following and obstacle avoidance robot.

C. Hardware Assembly

All components were mounted on a chassis and connected as per the schematic. For assembly, all of the parts were mounted using spax screws in 3 different lengths : 10mm, 16mm, 20mm with all of them being M3 diameter. Some

holes in the wooden chassis were done using a drill. All of the components were mounted without any adhesive materials. In our assembly, it was made sure that all components were firmly secured in position.

IV. Circuit Design

The autonomous vehicle circuit integrates multiple sensor and actuator subsystems using an Arduino Uno as the main controller. The complete circuit layout simulation in tinkercad is shown in Fig. 9.

The wiring was designed to fulfill three key goals: modularity, robustness, and safe power distribution. Each subsystem was treated as an independent module, allowing for easier debugging and incremental development.

A. Sensor Integration

- **Infrared Line Sensors (2x)** are connected to digital input pins D12 and D13. These sensors detect the black line against the white surface, and their values are read in real-time to determine the robot's lateral alignment.
- **Ultrasonic Sensors (2x)** are mounted on the front-left and front-right sides. Their trigger and echo pins are connected to D6/D7 (right) and D8/D9 (left). These provide real-time distance feedback to avoid collisions.
- **Color Sensor (1x)** is connected to analog pins A0 to A4. It adds redundancy to object classification, allowing the robot to treat red-colored obstacles differently (e.g., push or bypass).

B. Actuator Wiring

The circuit schematic of the Motor Driver along with the motors and ultrasonic sensors is shown in 10.

- **Motor Driver (L298N)** is used to control two DC motors. It is connected to:
  - PWM pins D10 and D11 for speed control
  - Direction control pins D2–D5

  The motors are powered directly from the 12V battery pack via the motor driver's onboard regulator.

C. Power Management

- A 12V DC battery pack supplies power to both motors and the Arduino.
- A voltage regulator onboard the L298N downscales the 12V to 5V for the Arduino and sensors, avoiding voltage spikes and protecting delicate components.
- Decoupling capacitors are placed across power lines to minimize electrical noise and ensure stable analog sensor readings.

D. Pin Mapping Overview

TABLE I
Key Pin Assignments

Component	Pin(s)
IR Sensor (Left)	D13
IR Sensor (Right)	D12
Ultrasonic Sensor (Left)	D8 (Trig), D9 (Echo)
Ultrasonic Sensor (Right)	D6 (Trig), D7 (Echo)
Left Motor	D2 (F), D3 (B), D10 (PWM)
Right Motor	D4 (F), D5 (B), D11 (PWM)
Color Sensor	A0–A4

E. Schematic Analysis

The circuit was simulated and verified using Tinkercad and real-world breadboard prototyping. Care was taken to route ground lines properly, avoid floating inputs, and apply pull-down resistors where needed.

The design emphasizes modularity, with all sensors and motors detachable via male/female header connectors for ease of maintenance and reassembly.

V. Software Implementation

A. Initialization of Sensors and Actuators

1. *Sensors Initialization*:

a) **Ultrasonic Sensors (Left/Right)** — Pins for the ultrasonic sensors are defined in the file 'UltrasonicSensor.h'. They are set up and initialized in the 'UltrasonicSensor.cpp' and 'main.cpp' files. Relevant snippets are shown below:

```
// Ultrasonic sensor pin definitions
#define LEFT_TRIG_PIN 8
#define LEFT_ECHO_PIN 9
#define RIGHT_TRIG_PIN 6
#define RIGHT_ECHO_PIN 7
```

Listing 1. Pin definitions in UltrasonicSensor.h

```
// Set trigger as output and echo as input
void UltrasonicSensor::begin() {
   pinMode(trigPin, OUTPUT);
   pinMode(echoPin, INPUT);
}
```

Listing 2. Pin setup in UltrasonicSensor.cpp

```
// Create hardware instances
...
UltrasonicSensor rightUltrasonic(
  RIGHT_TRIG_PIN,
  RIGHT_ECHO_PIN);
UltrasonicSensor leftUltrasonic(
  LEFT_TRIG_PIN,
  LEFT_ECHO_PIN);
RobotNavigation robotNav(
  &rightUltrasonic,
  &leftUltrasonic,
  &motor);

// Initialize ultrasonic sensors
void setup() {
   ...
   rightUltrasonic.begin();
   leftUltrasonic.begin();
```

```
  ...
}
```

Listing 3. Sensor initialization in main.cpp

b) **IR Sensors (Left/Right)** — Pins for the IR sensors are defined in 'RobotNavigation.h' and initialized in 'RobotNavigation.cpp' and in 'main.cpp'. Relevant snippets are shown below:

```
// Line sensor digital pins
#define LINE_SENSOR_LEFT 13
#define LINE_SENSOR_RIGHT 12
```

Listing 4. IR sensor pin definitions in RobotNavigation.h

```
void RobotNavigation::begin() {
  pinMode(LINE_SENSOR_LEFT, INPUT);
  pinMode(LINE_SENSOR_RIGHT, INPUT);
}
```

Listing 5. IR sensor pin initialization in RobotNavigation.cpp

```
void setup() {
  ...
  // Start navigation system
  robotNav.begin();
}
```

Listing 6. IR sensor pin initialization in main.cpp

c) **Color Sensor** — Pins for the color sensor are defined in 'ColorSensor.h' and initialized in 'ColorSensor.cpp'. Relevant snippets are shown below:

```
// Pin definitions
#define COLOR_SENSOR_OUTPUT_FREQUENCY_INPUT_S0
    ↪ A0
#define COLOR_SENSOR_OUTPUT_FREQUENCY_INPUT_S1
    ↪ A1
#define COLOR_SENSOR_PHOTODIODE_INPUT_S2 A2
#define COLOR_SENSOR_PHOTODIODE_INPUT_S3 A3
#define COLOR_SENSOR_OUTPUT_PIN A4
```

Listing 7. Color sensor pin definitions in ColorSensor.h

```
void ColorSensor::ColorSensorInit() {
    pinMode(
      COLOR_SENSOR_OUTPUT_FREQUENCY_INPUT_S0,
      OUTPUT);
    pinMode(
      COLOR_SENSOR_OUTPUT_FREQUENCY_INPUT_S1,
      OUTPUT);
    pinMode(
      COLOR_SENSOR_PHOTODIODE_INPUT_S2,
      OUTPUT);
    pinMode(
      COLOR_SENSOR_PHOTODIODE_INPUT_S3,
      OUTPUT);
    pinMode(
      COLOR_SENSOR_OUTPUT_PIN,
      INPUT);

    digitalWrite(
      COLOR_SENSOR_OUTPUT_FREQUENCY_INPUT_S0,
      HIGH); // Full scale
    digitalWrite(
      COLOR_SENSOR_OUTPUT_FREQUENCY_INPUT_S1,
      LOW); // Frequency scaling = 20%
}
```

Listing 8. Color sensor pin initialization in ColorSensor.cpp

2. Actuators (Motors) Initialization: Motor Control Pins are defined in the file 'MotorControl.h'. Their setup and initialization are made in the 'main.cpp' file. Relevant snippets are shown below:

```
// Motor control pin definitions
#define LEFT_MOTOR_SPEED_PIN 10
#define RIGHT_MOTOR_SPEED_PIN 11
#define LEFT_MOTOR_FORWARD_PIN 2
#define LEFT_MOTOR_BACKWARD_PIN 3
#define RIGHT_MOTOR_FORWARD_PIN 4
#define RIGHT_MOTOR_BACKWARD_PIN 5
```

Listing 9. Pin definitions in MotorControl.h

```
// Create hardware instances
MotorControl motor;
...

RobotNavigation robotNav(
  &rightUltrasonic,
  &leftUltrasonic,
  &motor
);
// Arduino initialization
void setup() {
  // Setup motor pins as outputs
  pinMode(LEFT_MOTOR_SPEED_PIN, OUTPUT);
  pinMode(RIGHT_MOTOR_SPEED_PIN, OUTPUT);
  pinMode(LEFT_MOTOR_FORWARD_PIN, OUTPUT);
  pinMode(LEFT_MOTOR_BACKWARD_PIN, OUTPUT);
  pinMode(RIGHT_MOTOR_FORWARD_PIN, OUTPUT);
  pinMode(RIGHT_MOTOR_BACKWARD_PIN, OUTPUT);
  ...
}
```

Listing 10. Pins initialization in main.cpp

Key Initialization Features:

1) **Pin Abstraction**
   - Hardware pins are abstracted via `#define` macros.
   - Logical separation of sensor and actuator types.
2) **Modular Initialization**
   - Each sensor type has its own initialization routine.
   - Motor control setup is centralized in `main.cpp`.
3) **Hardware–Software Mapping**
   - On Fig. 1, the block diagram shows how each hardware component is mapped to specific software-controlled pins in the Arduino controller.

This summary ensures the reader understands the overarching design—pin abstraction, modular setup, mapping, and your carefully ordered startup sequence—before diving into higher-level behavior logic.

B. Movement Control Algorithms

1. Core Movement Methods:

a) **Forward and Backward Motion** - The robot's linear motion is achieved through symmetric control of both motors using Pulse Width Modulation (PWM) for speed regulation. The direction of rotation is set by toggling the appropriate H-bridge control pins. The methods of the `MotorControl class` in 'MotorControl.cpp' show the implementation of forward and reverse movement:

```
// Move robot straight forward at given speed
void MotorControl::moveForward(int speed) {
  analogWrite(LEFT_MOTOR_SPEED_PIN, speed);
  analogWrite(RIGHT_MOTOR_SPEED_PIN, speed);
  digitalWrite(LEFT_MOTOR_FORWARD_PIN, LOW);
  digitalWrite(LEFT_MOTOR_BACKWARD_PIN, HIGH);
  digitalWrite(RIGHT_MOTOR_FORWARD_PIN, LOW);
  digitalWrite(RIGHT_MOTOR_BACKWARD_PIN, HIGH);
}

// Move robot straight backward at given speed
void MotorControl::moveBackward(int speed) {
  analogWrite(LEFT_MOTOR_SPEED_PIN, speed);
  analogWrite(RIGHT_MOTOR_SPEED_PIN, speed);
  digitalWrite(LEFT_MOTOR_FORWARD_PIN, HIGH);
  digitalWrite(LEFT_MOTOR_BACKWARD_PIN, LOW);
  digitalWrite(RIGHT_MOTOR_FORWARD_PIN, HIGH);
  digitalWrite(RIGHT_MOTOR_BACKWARD_PIN, LOW);
}
```

Listing 11. Forward and backward movement functions

Key Algorithm:

- PWM signals regulate motor speed via `analogWrite()`.
- H-bridge circuits manage direction through digital pins.
- Equal PWM ensures coordinated, straight-line movement.

b) **Pivotal Turning** - For medium-radius turns, one motor is activated while the other is disabled, allowing the robot to pivot around the unpowered wheel. The `turnLeft()` and `turnRight()` methods implement this by powering one motor while setting both control pins for the other motor to `LOW`, causing it to freewheel or coast. This creates a smoother turn compared to a point rotation.

```
// Turn robot to the left
void MotorControl::turnLeft(int speed) {
  analogWrite(LEFT_MOTOR_SPEED_PIN, speed);
  digitalWrite(LEFT_MOTOR_FORWARD_PIN, LOW);
  digitalWrite(LEFT_MOTOR_BACKWARD_PIN, HIGH);
  digitalWrite(RIGHT_MOTOR_FORWARD_PIN, LOW);
  digitalWrite(RIGHT_MOTOR_BACKWARD_PIN, LOW);
}

// Turn robot to the right
void MotorControl::turnRight(int speed) {
  analogWrite(RIGHT_MOTOR_SPEED_PIN, speed);
  digitalWrite(LEFT_MOTOR_FORWARD_PIN, LOW);
  digitalWrite(LEFT_MOTOR_BACKWARD_PIN, LOW);
  digitalWrite(RIGHT_MOTOR_FORWARD_PIN, LOW);
  digitalWrite(RIGHT_MOTOR_BACKWARD_PIN, HIGH);
}
```

Listing 12. Single-wheel pivot turn functions

Key Algorithm:

- Only one motor is actively powered for the turn.
- The other motor is disabled and its wheel is allowed to coast, serving as the pivot point.
- The turning radius is determined by the robot's geometry and the speed of the powered wheel.

c) **In-Place Rotation** - In-place rotation is accomplished by driving both motors in opposite directions at equal speed. This results in a zero turning radius, allowing the robot to pivot around its center. The technique is useful in constrained environments or for rapid heading adjustments.

```
// Rotate robot to the right
void MotorControl::rotateRight(int speed) {
  analogWrite(RIGHT_MOTOR_SPEED_PIN, speed);
  analogWrite(LEFT_MOTOR_SPEED_PIN, speed);
  digitalWrite(LEFT_MOTOR_FORWARD_PIN, HIGH);
  digitalWrite(LEFT_MOTOR_BACKWARD_PIN, LOW);
  digitalWrite(RIGHT_MOTOR_FORWARD_PIN, LOW);
  digitalWrite(RIGHT_MOTOR_BACKWARD_PIN, HIGH);
}

// Rotate robot to the left
void MotorControl::rotateLeft(int speed) {
  analogWrite(RIGHT_MOTOR_SPEED_PIN, speed);
  analogWrite(LEFT_MOTOR_SPEED_PIN, speed);
  digitalWrite(LEFT_MOTOR_FORWARD_PIN, LOW);
  digitalWrite(LEFT_MOTOR_BACKWARD_PIN, HIGH);
  digitalWrite(RIGHT_MOTOR_FORWARD_PIN, HIGH);
  digitalWrite(RIGHT_MOTOR_BACKWARD_PIN, LOW);
}
```

Listing 13. Counter-rotational in-place turning

Key Algorithm:

- Opposite motor directions produce a central rotation.
- Equal PWM ensures balanced torque and pivoting.
- Enables highly responsive directional changes.

d) **Stopping Mechanism** - To halt motion, both motors are actively short-braked by setting both control lines to `HIGH`. Simultaneously, PWM is set to its maximum to ensure full coil engagement. This braking approach ensures faster and more stable stops compared to passive coasting.

```
// Stop robot
void MotorControl::stopMotors() {
  analogWrite(LEFT_MOTOR_SPEED_PIN, 255);
  analogWrite(RIGHT_MOTOR_SPEED_PIN, 255);
  digitalWrite(LEFT_MOTOR_FORWARD_PIN, HIGH);
  digitalWrite(LEFT_MOTOR_BACKWARD_PIN, HIGH);
  digitalWrite(RIGHT_MOTOR_FORWARD_PIN, HIGH);
  digitalWrite(RIGHT_MOTOR_BACKWARD_PIN, HIGH);
}
```

Listing 14. Active braking mechanism

Key Algorithm:

- Short-circuit braking is applied to both motors.
- Ensures rapid deceleration and positional stability.
- Full PWM maximizes current for immediate response.

2. Parameterization and Tuning: To balance power consumption, control responsiveness, and maneuverability, a set of predefined constants are used for speed tuning. These constants are defined in the header file `RobotNavigation.h` and used throughout the motion control routines.

```
// Speed configurations
#define DefaultTurnSpeed 140
#define DefaultRotateSpeed 120
#define DefaultForwardSpeed 120
#define DefaultBackwardSpeed 80
```

Listing 15. Speed tuning parameters

Tuning Principles:

- **High Turn Speed:** The turning speed (140) is set higher than the forward speed to ensure rapid and responsive correction when the robot deviates from the line.
- **Controlled Rotation and Forward Motion:** Forward and rotation speeds (120) are balanced to provide stable movement during standard navigation and obstacle avoidance maneuvers.
- **Reduced Reverse Speed:** Reverse motion uses a lower PWM value (80) for increased precision and safety, particularly when backing away from an obstacle.
- **PWM Range:** All values are calibrated within the 0–255 PWM range to prevent motor saturation while providing adequate torque.

3. Real-World Execution Flow: The robot's low-level movement primitives are integrated with sensor feedback to execute complex, high-level behaviors. The obstacle avoidance routine is a key example, combining multiple motion types in a timed and sensor-driven sequence.

```
// Behavior for obstacle avoidance sequence
void RobotNavigation::handleObstacleAvoidance() {
  switch (obstacleStep) {
    // Move backward to clear way for maneuver
    case 0:
      motor->stopMotors();
      delay(100);
      motor->moveBackward(DefaultBackwardSpeed);
      delay(200);
      obstacleStep++;
      break;
    // Rotate left before bypass
    case 1:
      motor->rotateLeft(80);
      delay(600);
      motor->stopMotors();
      delay(1000);
      obstacleStep++;
      break;
  }

  // Check obstacle on the left side
  float rightDistance = rightSensor->getDistance();
  float leftDistance = leftSensor->getDistance();

  // Scenario 1: Obstacle detected
  if ((rightDistance < 100 && rightDistance > 3) ||
      (leftDistance < 100 && leftDistance > 3)) {

    /* Rotate right to recover straight position.
    The while loop allows to find line back by
    rotating right until line is detected again */

    motor->rotateRight(68);
    while (true) {
      bool leftLine = digitalRead(LINE_SENSOR_LEFT);
      bool rightLine =
          ↪ digitalRead(LINE_SENSOR_RIGHT);
      if (rightLine == HIGH) {
        motor->stopMotors();
        delay(500);
        break;
      }
    }

    /* Once recovered the straight position,
    rotate right to bypass from the right side
    which is clear from obstacles */

    motor->rotateRight(75);
    delay(500);

    /* Bypass the obstacle by executing a pre-tuned
    multi-step maneuver, developed through testing
    with the specific hardware and environment
    conditions */

  // Scenario 2: Obstacle is not detected
  } else {

    // Rotate slightly right
    motor->rotateRight(75);
    delay(350);

    /* Execute bypass with pre-tuned multi-step
    maneuver, developed through testing */
  }

  /* In the end of bypass, switch to
  LINE_SEARCH to reacquire the path */
  currentState = LINE_SEARCH;
  setRobotState(LINE_SEARCH);
}
```

Listing 16. Excerpt from the Obstacle Avoidance Sequence

This procedure shows how low-level primitives (`moveBackward`, `rotateLeft`, etc.) are composed into a stateful, adaptive sequence. By layering control logic over robust actuator routines, the system achieves both predictability and adaptability in dynamic environments.

C. Behavioral Decision Logic

The robot's autonomy is governed by a state machine architecture implemented in the `RobotNavigation class`. This design ensures that the robot transitions between well-defined behaviors based on sensor inputs. The logic is visualized in the state machine diagram in Fig 7 and implemented in the methods below.

1. State Machine Architecture: The core of the decision-making process is located in the `updateNavigation()` method in 'RobotNavigation.cpp', which runs continuously. It performs the reading of all relevant sensors and the usage of a prioritized decision tree to select the robot's next state.

a) **Core States** - The primary operational states are defined as macros in 'RobotNavigation.h'.

```
// Robot states
#define OBSTACLE_AVOID 0
#define LEFT 1
#define RIGHT 2
#define LINE_SEARCH 3
```

Listing 17. State Definitions in RobotNavigation.h

b) **State Transition Logic** - The state transition logic is based on a priority-based sequence implemented in the `updateNavigation()` method, where the obstacle avoidance is the highest priority.

```
void RobotNavigation::updateNavigation() {
  // Read line and obstacle sensors
  int leftLine = digitalRead(LINE_SENSOR_LEFT);
  int rightLine =
      ↪ digitalRead(LINE_SENSOR_RIGHT);
  float rightDistance =
      ↪ rightSensor->getDistance();
```

```
  float leftDistance =
      ↪ leftSensor->getDistance();

  // Decision Tree
  // Priority 1: Obstacle detection
  if (((rightDistance < maxDistance &&
        rightDistance > minDistance) ||
       (leftDistance < maxDistance &&
        leftDistance > minDistance)) &&
       !red->ColorSensorObserve()) {
    motor->stopMotors();
    delay(300);
    setRobotState(OBSTACLE_AVOID);
  }

  // Priority 2: Line following

  // Right sensor lost the line
  if (leftLine == HIGH && rightLine == LOW) {
    motor->stopMotors();
    setRobotState(LEFT);  // Turn left

  // Left sensor lost the line
  } else if (leftLine == LOW && rightLine ==
      ↪ HIGH) {
    motor->stopMotors();
    setRobotState(RIGHT);  // Turn right

  // Both sensors lost the line
  } else if (leftLine == LOW && rightLine ==
      ↪ LOW) {
    // Invert last state
    if (currentState == LEFT) {
      motor->stopMotors();
      setRobotState(RIGHT);
    } else if (currentState == RIGHT) {
      motor->stopMotors();
      setRobotState(LEFT);
    }
  }
}
```

Listing 18. Decision Logic in RobotNavigation.cpp

An important feature of the decision logic is the integration of the color sensor. The robot should only enter the `OBSTACLE_AVOID` state if the detected object is `NOT RED`, allowing it to push red objects off the line and continue following it.

2. State-Specific Behaviors: Once a state is set via `setRobotState()` method, a specific handler function is executed.

a) **OBSTACLE_AVOID State** — This is a multi-step routine managed by the `obstacleStep` attribute of the `RobotNavigate` class. It does not execute in a single function call but progresses through a sequence.

- **Step 0 & 1:** The robot first moves backward to create space, then rotates to face a new heading.
- **Conditional Bypass:** After the initial turn, the robot re-checks for another obstacle on the left. If one is detected within a 3-100 cm range, a longer, more complex bypass maneuver is executed: the robot first rotates right to recover its original position using the line sensors, and then rotates right again to initiate a bypass from the opposite side, which is presumed clear. Otherwise, a shorter maneuver is performed to return to the path (see Fig. 7 and the code snippet in Listing 16)
- **Exit Condition:** Upon completing the bypass maneuver, the robot transitions to the `LINE_SEARCH` state to reacquire the line.

b) **LEFT and RIGHT States** — These states handle the line-following course. The robot executes a pivot turn until the opposite sensor rediscovers the line, ensuring the robot straightens out. This logic applies a zig-zag pattern, allowing the robot to follow the line even through the sharpest curves.

```
case LEFT:
  /* Turn left until the right IR sensor
  detects the line */

  while (digitalRead(LINE_SENSOR_RIGHT) == LOW)
      motor->turnLeft(DefaultTurnSpeed);
  motor->stopMotors();
  break;

case RIGHT: // The RIGHT case is symmetric
...
```

Listing 19. Line Correction Logic

c) **LINE_SEARCH State** — This state is triggered after the robot completes an obstacle avoidance maneuver. The logic of this state is implemented in the `handleLineSearch()` method in 'RobotNavigation.cpp'. The method executes a zig-zag pattern, first turning left and then right in small increments, until one of the sensors finds the line.

- **Timeout Failsafe:** This search is constrained by a `searchTimeout` of 100,000 ms. If the line is not found within this period, the routine terminates, which corresponds to the transition to the final `STOPPED` state in the state diagram (see Fig. 7).
- **Re-alignment:** Once the line is detected, a final adjustment is made to center the robot before resuming forward motion.

```
void RobotNavigation::handleLineSearch() {
  bool foundLine = false;
  unsigned long startTime = millis();
  while (millis() - startTime < searchTimeout) {
    motor->turnLeft(120);
    delay(110);
    motor->stopMotors();
    delay(100);

    bool leftLine = digitalRead(LINE_SENSOR_LEFT);
    bool rightLine = digitalRead(LINE_SENSOR_RIGHT);

    if (leftLine == HIGH || rightLine == HIGH) {
        // Realignment sequence
        if (leftLine == HIGH)
            while (rightLine == HIGH)
                motor->turnLeft(DefaultTurnSpeed);

        if (rightLine == HIGH)
            while (leftLine == HIGH)
                motor->turnRight(DefaultTurnSpeed);

        break;
    }
```

```
    // Try right turn if line not found and repeat
    motor->turnRight(DefaultTurnSpeed);
    delay(100);
    motor->stopMotors();
    delay(100);
    ...
}
```

Listing 20. Excerpt from the Line Search Sequence

3. Behavioral Characteristics Summary: The Table III summarizes the primary state transitions. The ”Default” behavior of moving forward is the implicit action when no other trigger condition is met.

VI. Git Usage and Collaboration

Project development was tracked using GitHub. The following Table II, shows each member and number of commits made to the git Repository.

Team Member	**Number of Commits**
Mohamed Amer	30
Daria Shushkova	72
Andrey Sysoev	58
Sadim Rahman	34
Total	**194**

TABLE II
GitHub Commits by Team Members

VII. Key Achievements

Our autonomous car was able to follow the line smoothly, handling sharp turn at high speeds, successfully detecting obstacles and avoiding them, all of this while keeping a good accuracy of the line tracking. Moreover, we were successful in achieving successful integration of the hardware, consistency of results across multiple tests, effective collaboration using Git for version control and systematic testing and debugging approach.

Acknowledgment

References

[1] Arduino Documentation. https://www.arduino.cc/en/Guide
[2] Tinkercad Simulation. https://www.tinkercad.com
[3] Uppaal Model Checker. http://www.uppaal.org/

Declaration of Originality

We hereby declare that this report is our own work and that we have acknowledged all sources used.

APPENDIX
SYSML DIAGRAMS

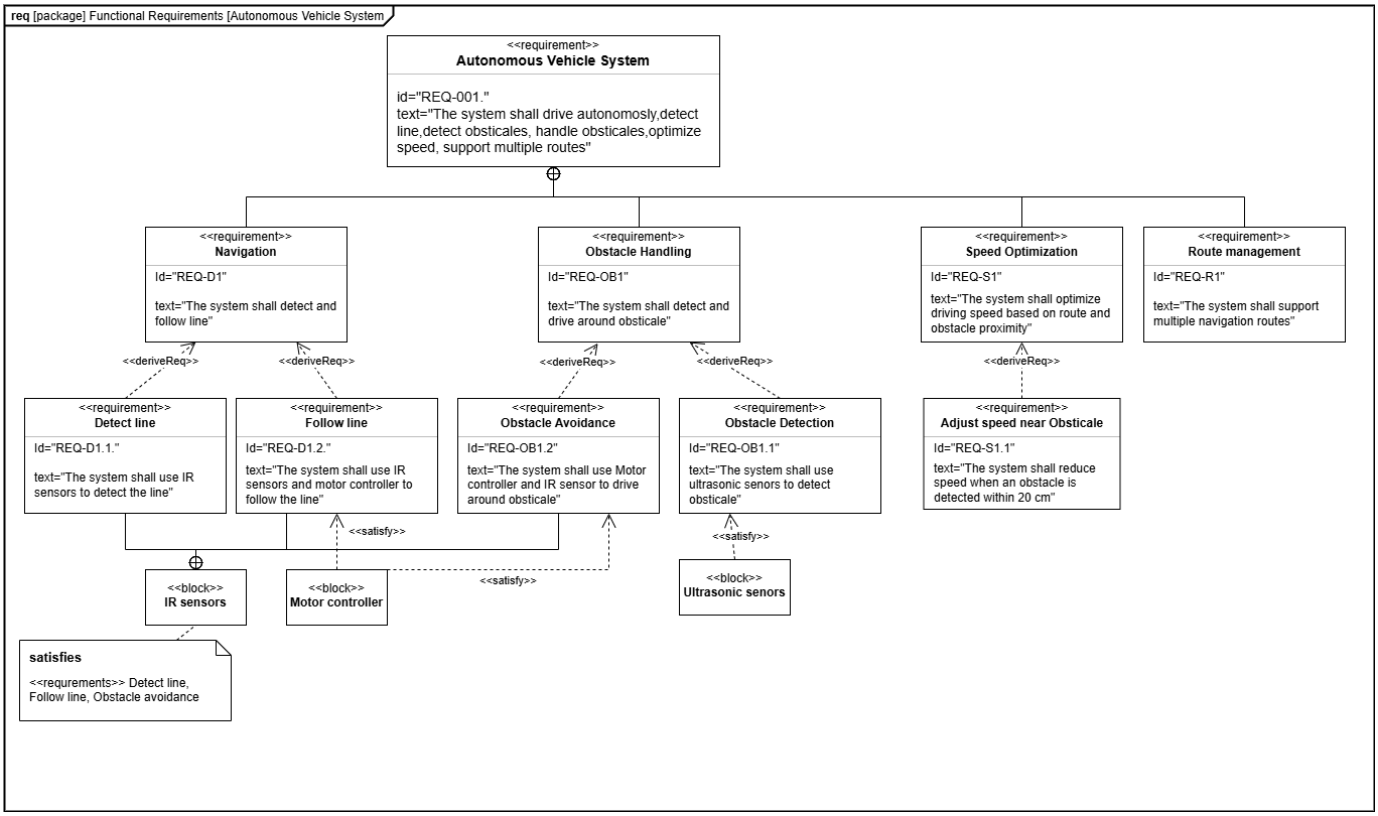

Fig. 3. Requirement diagram

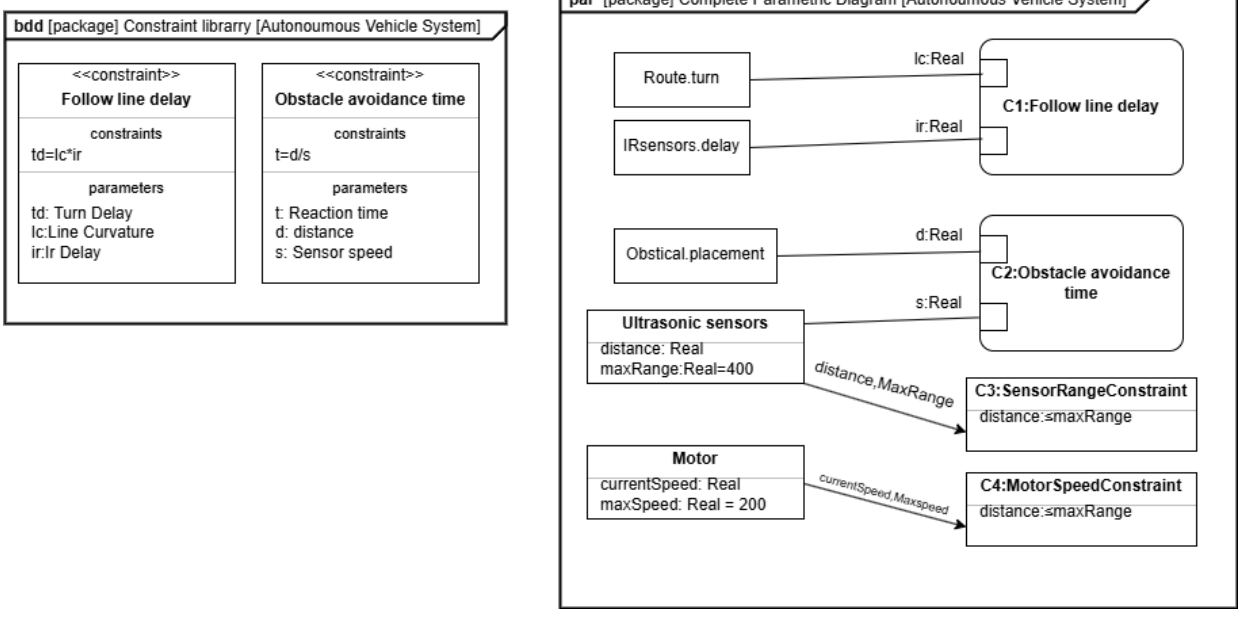

Fig. 4. Parametric diagram

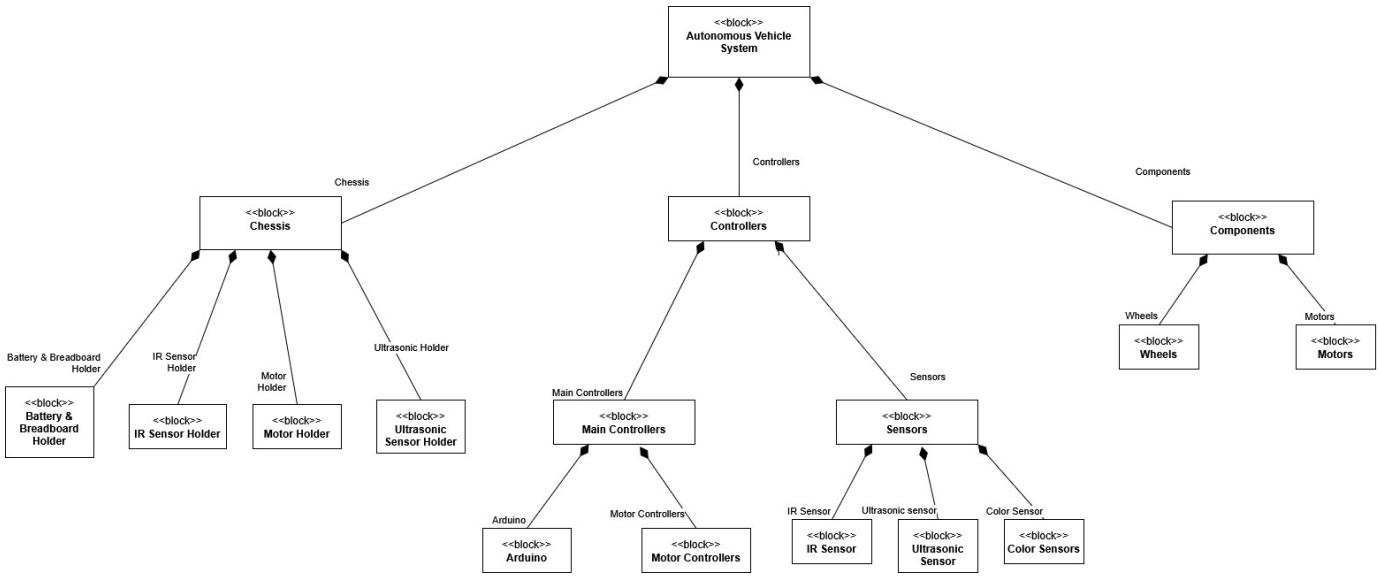

Fig. 5. Block Definition diagram

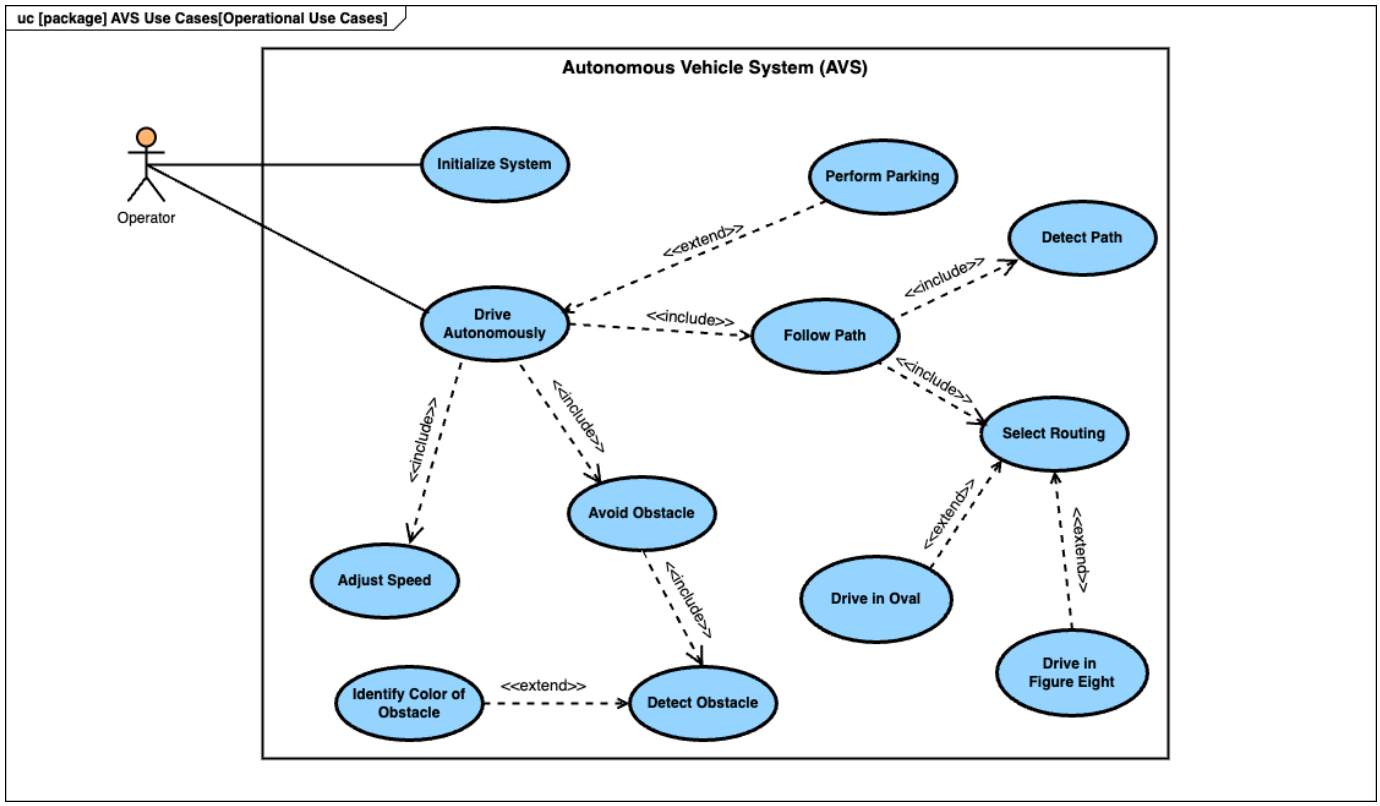

Fig. 6. Use Case diagram

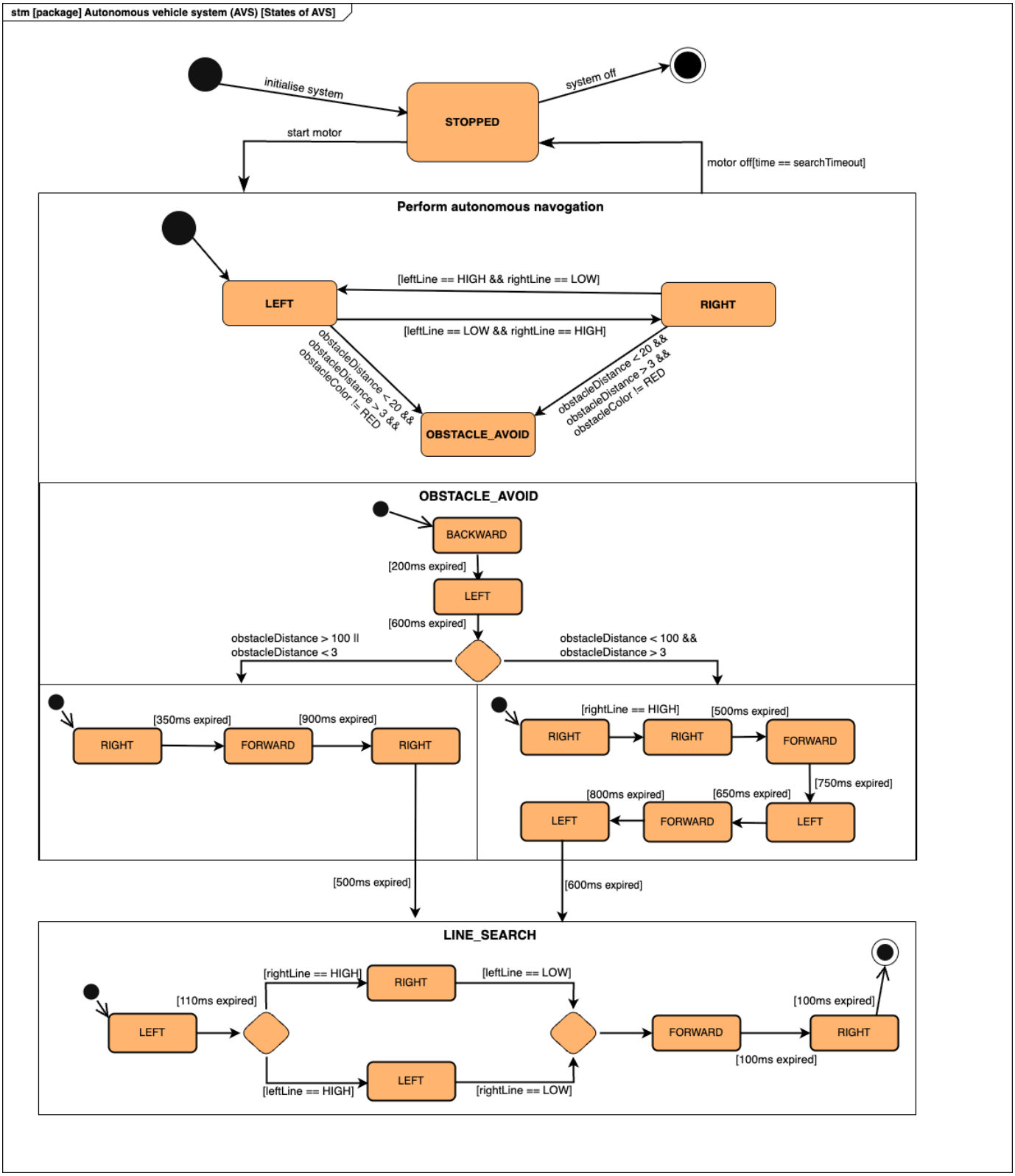

Fig. 7. State Machine diagram

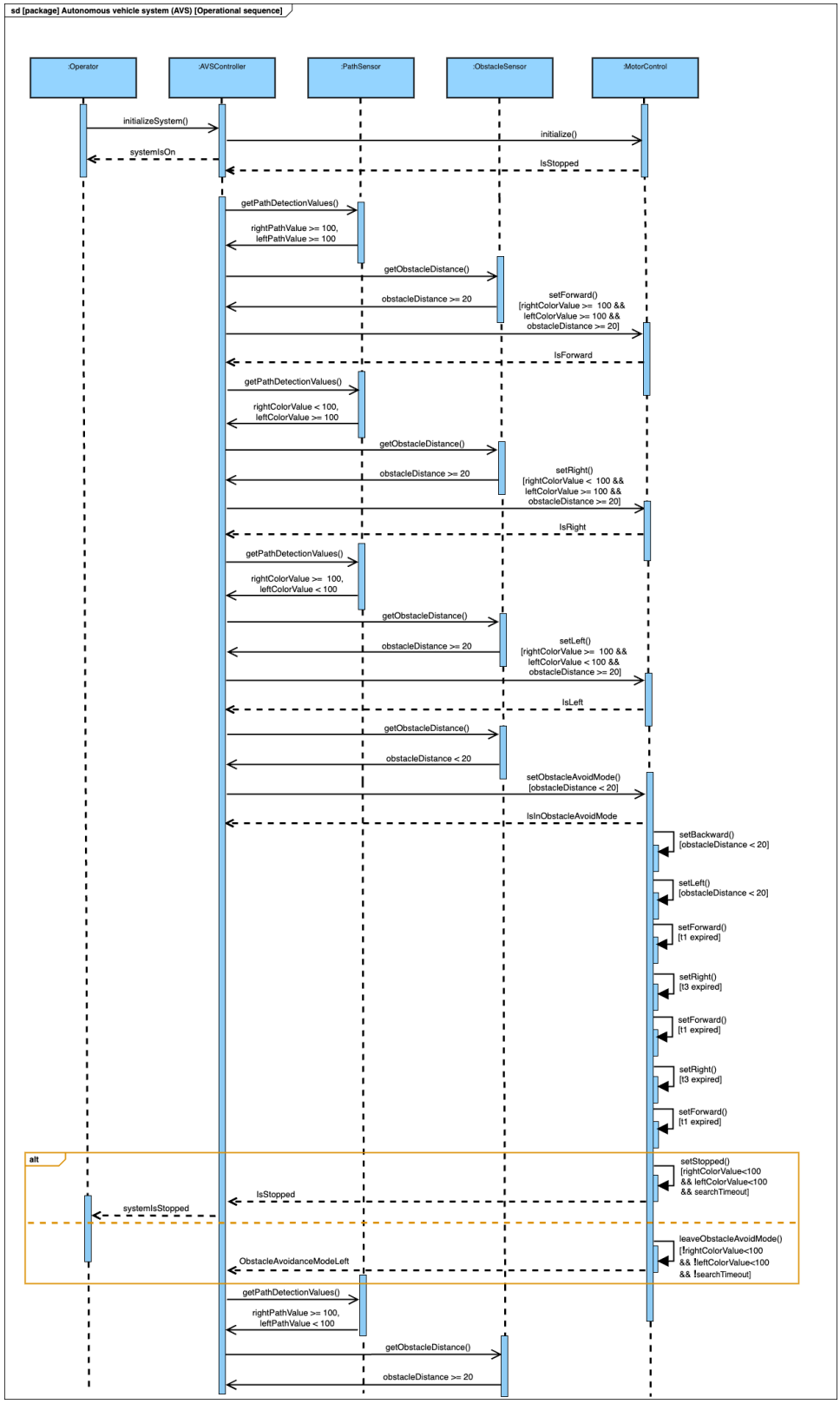

Fig. 8. Sequence diagram

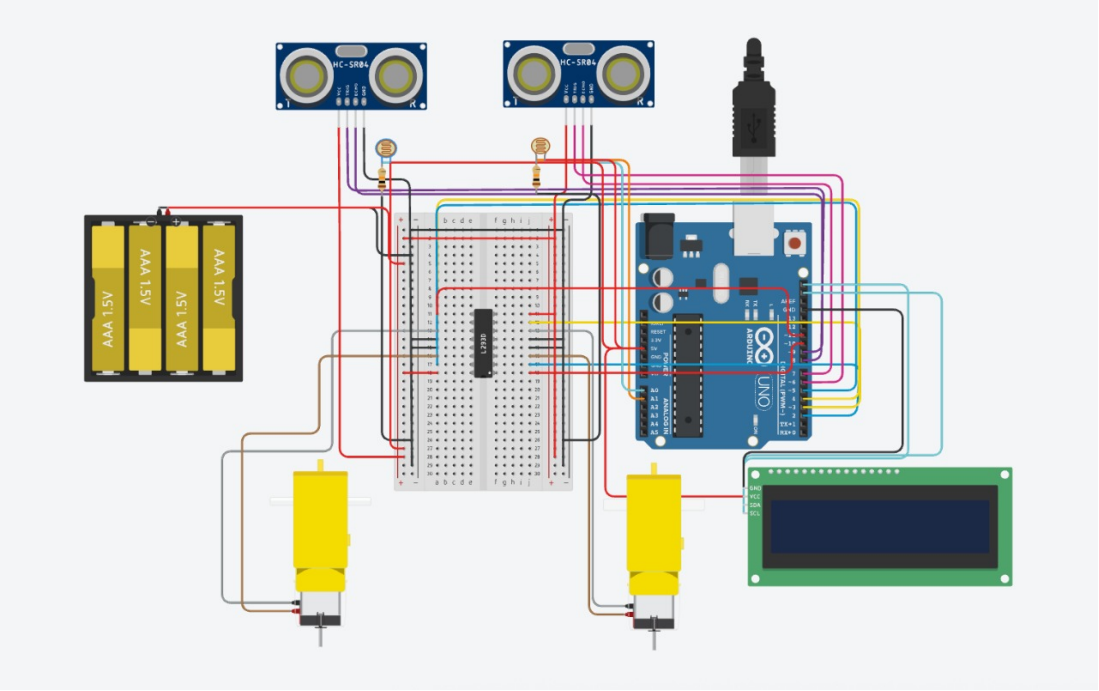

Fig. 9. Tinker Cad Simulation showing a complete circuit layout

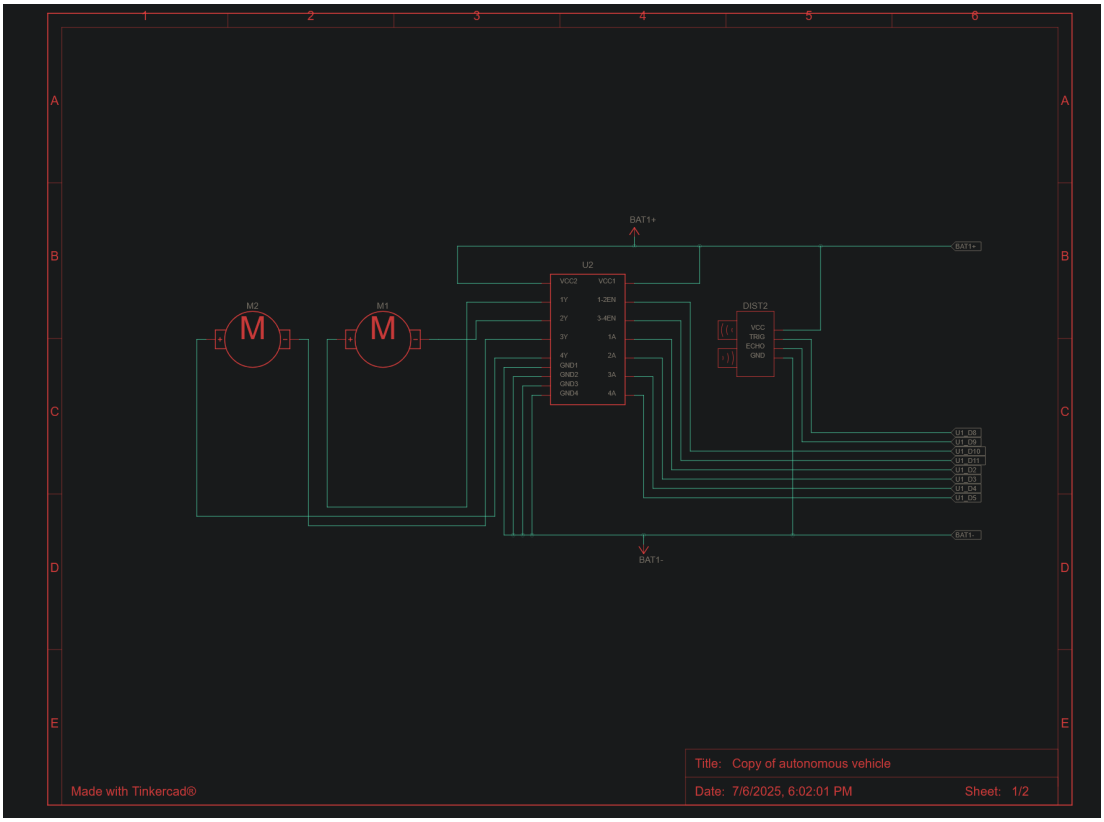

Fig. 10. Schematic for motor driver connection along with arduino, motors, ultrasonic sensors

TABLE III
ROBOT STATE BEHAVIOR SUMMARY

State	Trigger Condition	Primary Action	Exit Condition
`LEFT`	Left sensor on line	`turnLeft()`	Right sensor on line
`RIGHT`	Right sensor on line	`turnRight()`	Left sensor on line
`OBSTACLE_AVOID`	Object in 3–20 cm and not red	Multi-step bypass sequence	Maneuver completion
`LINE_SEARCH`	Both sensors off line	Scan left and right for the line	Line reacquired or timeout expires

Electronic Components	Description
Ultrasonic Sensor (HC-SR04)	Used for detecting obstacle distance
Motor Driver (L298N)	Used for stepping down voltage and operating motors
Infrared sensors (KY-033)	Used for detecting black lines
Color sensor (TCS3200)	Used for detecting obstacle color
Arduino Uno R4	used as the brain of the vehicle
Conrad LiPo battery 3200mah	Used to supply power to all componments

TABLE IV
A QUICK BRIEF INFO ABOUT THE SUPPLIED ELECTRONIC COMPONENTS